

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Antonia Contreras

11 de septiembre de 2026

21:14

1. Abstract

En este trabajo se realiza un análisis experimental del rendimiento práctico de algoritmos de ordenamiento (SORT, QUICKSORT, MERGESORT y PATIENCESORT) y de multiplicación de matrices cuadradas (NAIVE y STRASSEN), contrastando sus tiempos de ejecución y consumo de memoria con su complejidad asintótica teórica. Evaluados bajo diversas dimensiones, distribuciones iniciales y dominios numéricos, los resultados confirman la validez asintótica teórica y evidencian cómo las constantes ocultas, determinan la eficiencia real de los algoritmos.

2. Introducción

Las notaciones asintóticas permiten clasificar la eficiencia de los algoritmos acotando su tiempo de ejecución y requerimiento de memoria en función del tamaño de entrada. Sin embargo, estos modelos teóricos simplifican el comportamiento práctico al omitir las constantes multiplicativas. En este trabajo se analiza experimentalmente el desempeño de cuatro algoritmos de ordenamiento (SORT, QUICKSORT, MERGESORT y PATIENCESORT) y dos de multiplicación de matrices cuadradas (NAIVE y STRASSEN). El objetivo consiste en contrastar sus cotas teóricas conocidas con los resultados obtenidos de tiempo y memoria frente a variaciones en la disposición de los datos, el dominio numérico y la dimensión de las entradas.

3. Entorno experimental

Las mediciones experimentales se realizaron en un entorno Linux (Ubuntu 24.04.1 LTS sobre WSL2, arquitectura x86_64), en una máquina equipada con un procesador Intel Core i7-1255U (10 núcleos, 12 hilos, con frecuencias de hasta 4.70 GHz) y 6.6 GiB de memoria RAM asignados al entorno virtualizado. Las soluciones fueron desarrolladas en C++17 y compiladas mediante g++ versión 13.3.0 aplicando la bandera de optimización -O3. La medición temporal se instrumentó con `std::chrono::steady_clock` con resolución de microsegundos (μs), mientras que el consumo de memoria máxima residente se obtuvo a través de la llamada al sistema `getrusage (ru_maxrss)`. Para evitar contaminación acumulativa de memoria entre corridas, cada ejecución individual se ejecutó como un proceso independiente orquestado desde el `makefile`.

Los experimentos se evaluaron sobre las instancias de prueba parametrizadas en el enunciado:

- **Ordenamiento de Arreglos:** Arreglos de dimensión $n \in \{10^1, 10^3, 10^5, 10^7\}$, bajo tres tipos de orden previo (*aleatorio*, *ascendente* y *descendente*) y dos rangos de dominio: $D_1 = \{0, \dots, 9\}$ y $D_7 = \{0, \dots, 10^7\}$, evaluando 3 muestras independientes (a, b, c) por cada combinación (288 ejecuciones por algoritmo).
- **Multiplicación de Matrices:** Pares de matrices cuadradas de orden $n \in \{2^4, 2^6, 2^8, 2^{10}\}$, abarcando configuraciones *densas*, *diagonales* y *dispersas* (10% de elementos no nulos), con coeficientes pertenecientes a los dominios $D_0 = \{0, 1\}$ y $D_{10} = \{0, \dots, 9\}$, considerando 3 repeticiones independientes por instancia.

4. Resultados y Análisis

4.1. Memoria de Algoritmos de Sorting

El análisis experimental de la memoria confirma las cotas asintóticas y exhibe el peso de las estructuras de datos auxiliares. Teóricamente, QUICKSORT y SORT son algoritmos *in-place* con espacio auxiliar en la pila de recursión de $O(\log n)$; MERGESORT no es *in-place* al requerir un arreglo temporal continuo de dimensión idéntica a la entrada ($\Theta(n)$); y PATIENCESORT almacena la totalidad de los datos en pilas intermedias, conservando una cota espacial de $O(n)$.

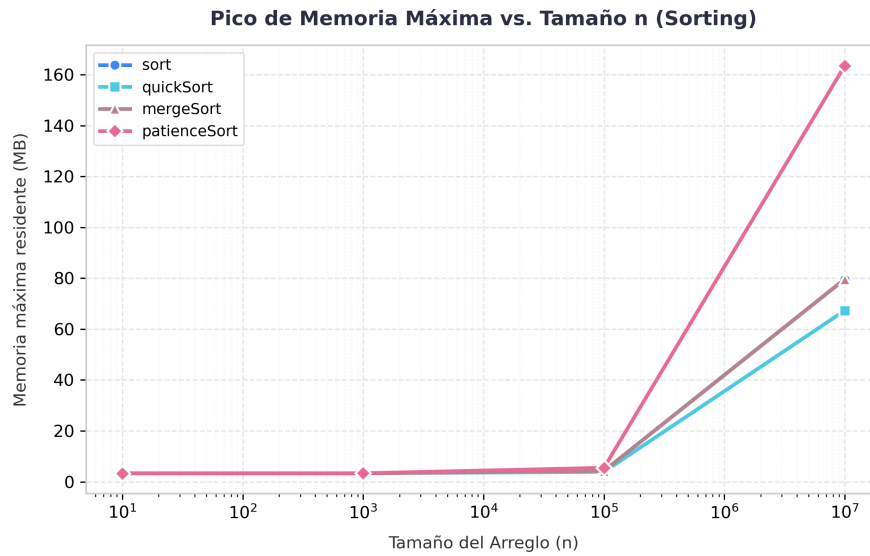


Figura 1: Pico de memoria máxima vs. tamaño del arreglo n en algoritmos de ordenamiento.

Como se aprecia en la Fig. 1, el consumo de memoria se mantiene bajo y uniforme para todos los algoritmos en dimensiones pequeñas ($n \leq 10^5$), disparándose de forma notoria al alcanzar $n = 10^7$. En esta escala, PATIENCESORT resulta ser el método menos eficiente: aunque su cota asintótica de datos es lineal $O(n)$, la representación en C++ mediante vectores dinámicos anidados aumenta el consumo de memoria respecto al resto. Por su parte, QUICKSORT se posiciona en el extremo más eficiente al operar directamente sobre el arreglo original. Finalmente, SORT y MERGESORT se ubican en una franja intermedia muy próxima entre sí; mientras que en MERGESORT esto concuerda con la reserva física de su vector auxiliar $\Theta(n)$, en SORT responde a la memoria base del proceso y a las asignaciones internas del entorno en tiempo de ejecución.

4.2. Tiempo de Ejecución de Algoritmos de Sorting

En el plano temporal, SORT garantiza $O(n \log n)$ en el peor caso; QUICKSORT exhibe un caso promedio de $O(n \log n)$ con peor caso $O(n^2)$; MERGESORT presenta una complejidad estricta de $\Theta(n \log n)$ en todos los escenarios; y PATIENCESORT posee una complejidad adaptativa de $O(n \log p)$, donde p denota la cantidad de pilas formadas ($1 \leq p \leq n$), fluctuando entre $O(n)$ en el mejor caso y $O(n \log n)$ en el peor caso.

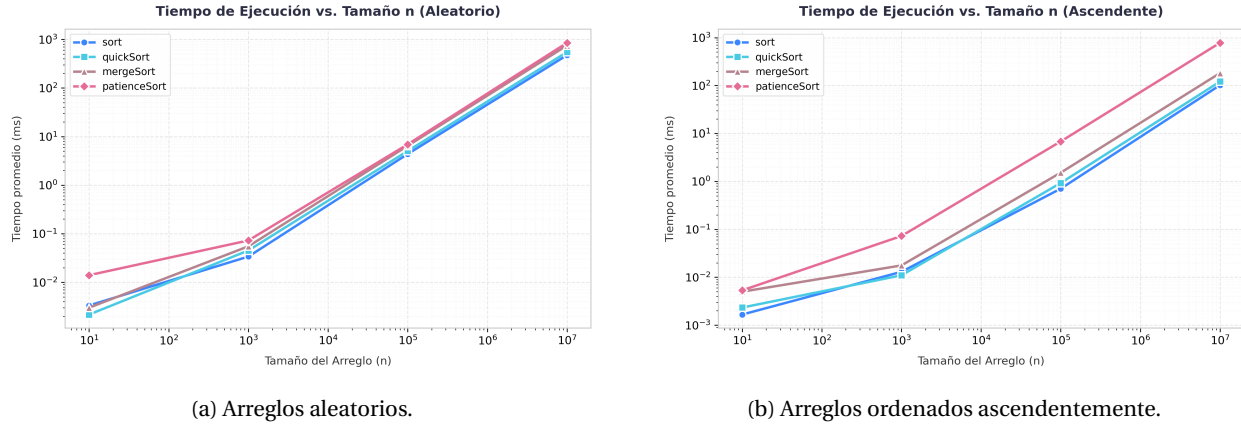


Figura 2: Tiempo de ejecución promedio vs. tamaño n para distintas distribuciones iniciales.

Al evaluar las instancias aleatorias y ascendentes (Fig. 2), los cuatro algoritmos presentan curvas con pendientes similares en escala log-log, confirmando que todos escalan según la familia $n \log n$. No obstante, PATIENCESORT se posiciona de manera constante como el algoritmo menos veloz. Esto se debe a que la dispersión y crecimiento de los elementos fuerza la creación de un número masivo de pilas ($p \approx n$), saturando la búsqueda binaria y encareciendo las operaciones en la cola de prioridad. En contraste, QUICKSORT y SORT se mantienen como los más rápidos gracias a su naturaleza *in-place*, la cual los hace mas rapidos al no tener que transferir datos a un arreglo auxiliar como hace MERGESORT.

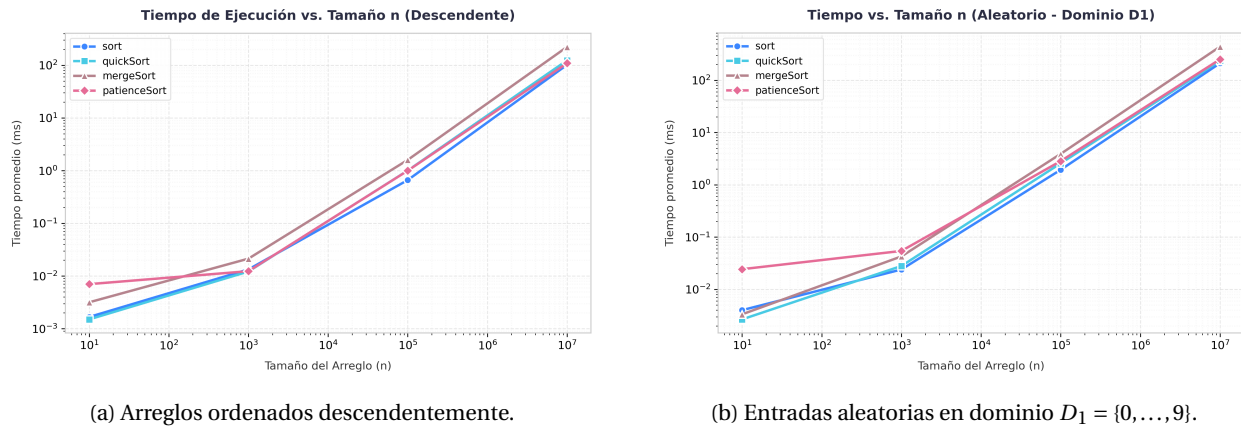


Figura 3: Comportamiento adaptativo de los algoritmos de ordenamiento frente al caso favorable y a la variación del dominio numérico.

La naturaleza adaptativa de PATIENCESORT se verifica empíricamente en la Fig. 3a. En arreglos con orden descendente, cada nuevo elemento resulta estrictamente menor que el tope anterior, acumulándose la totalidad de la entrada en una única pila ($p = 1$). Esto suprime el uso intensivo del heap y permite extraer la secuencia de forma lineal, logrando que la curva de PATIENCESORT descienda drásticamente hasta alcanzar la velocidad de los métodos *in-place*, en concordancia con su mejor cota teórica $O(n)$. Por

su parte, la Fig. 3b demuestra el impacto del dominio: en D_1 , al contar con solo diez valores posibles, la cantidad de pilas queda restringida a $p \leq 10$, lo que aplana sustancialmente la curva de PATIENCESORT y lo sitúa notablemente más veloz que MERGESORT.

4.3. Memoria de Algoritmos de Multiplicación de Matrices

Para el producto de matrices cuadradas de orden $n \times n$, NAIVE opera con una memoria auxiliar constante de $O(1)$, requiriendo únicamente el espacio cuadrático $\Theta(n^2)$ destinado a almacenar la matriz solución. En contraparte, STRASSEN aplica dividir y conquistar desglosando la matriz en cuatro cuadrantes de orden $n/2$ y generando siete matrices intermedias ($P_1 \dots P_7$) por cada nivel de llamada; aunque la convergencia de la serie espacial teórica mantiene una cota de $O(n^2)$, introduce una constante de asignación muy superior.

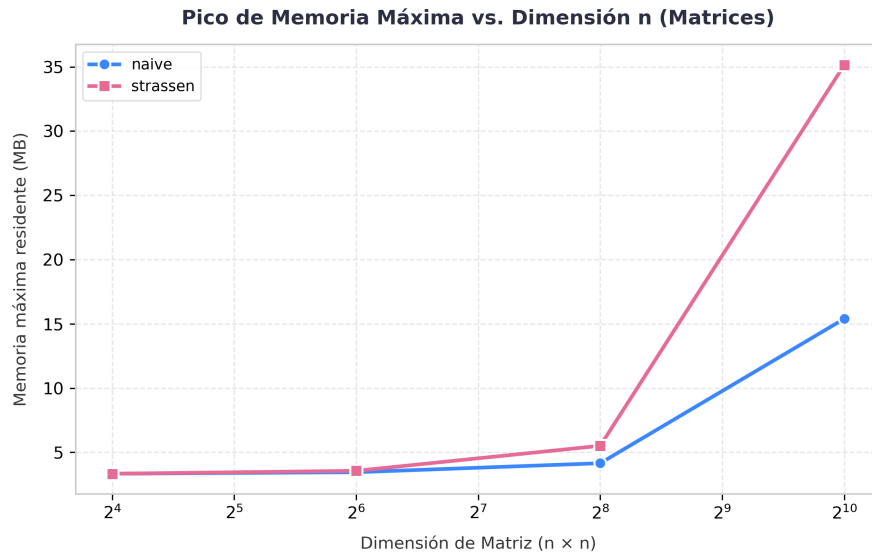


Figura 4: Pico de memoria máxima vs. dimensión n en multiplicación de matrices.

En la Fig. 4 se constata que STRASSEN utiliza considerablemente más memoria que NAIVE, incrementándose esta brecha conforme aumenta la dimensión n . Mientras que NAIVE exhibe un consumo plano y predecible al no demandar estructuras auxiliares durante el recorrido de sus bucles, STRASSEN acumula múltiples submatrices dinámicas creadas en el *heap* durante la recursión, confirmando que las constantes ocultas de asignación dinámica dominan el consumo físico del sistema.

4.4. Tiempo de Ejecución de Algoritmos de Multiplicación de Matrices

Teóricamente, el método iterativo clásico NAIVE ejecuta tres bucles anidados con n^3 multiplicaciones escalares, fijando una cota estricta de $\Theta(n^3)$. Por su parte, la complejidad de STRASSEN se deduce a partir de su ecuación de recurrencia $T(n) = 7T(n/2) + \Theta(n^2)$, asociada a la ejecución de 7 productos recursivos y operaciones de adición/sustracción de bloques en tiempo cuadrático. Aplicando el Teorema Maestro,

los parámetros $a = 7$ y $b = 2$ determinan el exponente crítico $\log_2 7 \approx 2,807$; dado que $f(n) = \Theta(n^2) = O(n^{\log_2 7 - \epsilon})$ para $\epsilon \approx 0,807$, rige el Caso 1, concluyendo una cota teórica asintótica de $\Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$.

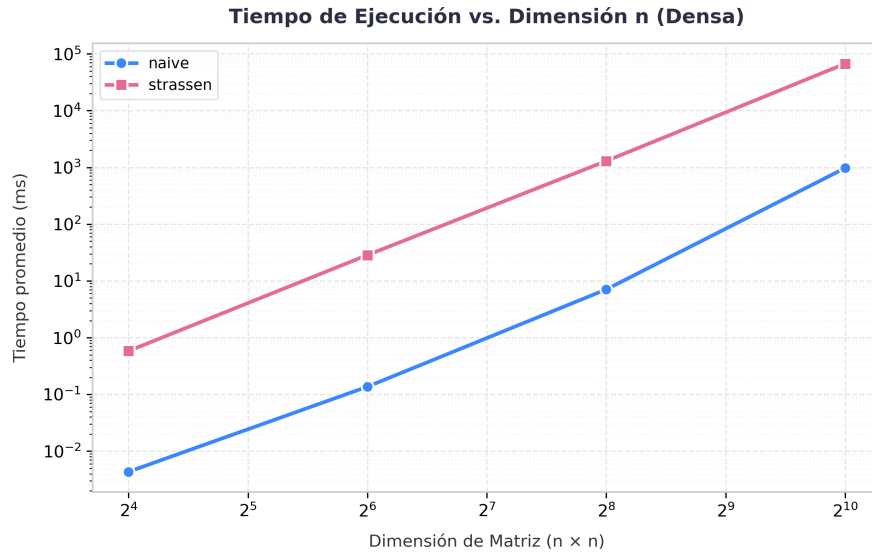


Figura 5: Tiempo de ejecución promedio vs. dimensión n en matrices densas.

A pesar de que la cota de STRASSEN ($n^{2,807}$) es asintóticamente inferior a la de NAIVE (n^3), en la [Fig. 5](#) se observa que STRASSEN resulta menos eficiente en la práctica para las dimensiones evaluadas. Esto no invalida la teoría, sino que evidencia el impacto de las constantes multiplicativas omitidas por la notación asintótica. STRASSEN ahorra productos de submatrices a expensas de introducir numerosas sumas y restas intermedias, sumado a la sobrecarga fija de instanciar vectores en el *heap* y apilar llamadas recursivas. Para dimensiones $n \leq 1024$, este costo administrativo continuo supera el beneficio asintótico, permitiendo que la ejecución lineal y secuencial de bucles anidados en NAIVE aproveche mejor la caché y domine temporalmente en el hardware evaluado.

5. Conclusiones

Los experimentos realizados confirmaron que la teoría asintótica predice bien el crecimiento general de los algoritmos, pero no refleja por sí sola el rendimiento real en la práctica, donde la gestión de memoria y la sobrecarga del código marcan una gran diferencia.

En el ordenamiento, el algoritmo más eficiente fue QUICKSORT, logrando los menores tiempos de ejecución y el uso más bajo de memoria al operar directamente sobre el arreglo original sin requerir estructuras adicionales. Por el contrario, el menos eficiente de forma general fue PATIENCESORT, que gastó mucha más memoria y tiempo debido a la sobrecarga de crear múltiples vectores dinámicos anidados en el heap. No obstante, este último mostró una notable ventaja adaptativa en arreglos ordenados de forma descendente y con dominios acotados (D_1), donde formó muy pocas pilas y alcanzó una rapidez lineal $O(n)$.

En la multiplicación de matrices, el método tradicional NAIVE superó con claridad a STRASSEN en matrices densas de hasta 1024×1024 . Aunque la cota teórica favorece a STRASSEN con $\Theta(n^{2,807})$ frente al $\Theta(n^3)$ de NAIVE, el costo continuo de dividir la matriz en tantas llamadas recursivas y crear submatrices auxiliares anula el ahorro matemático. En conclusión, una solución con mejor cota teórica no siempre es la más rápida si su costo estructural supera al de una alternativa más simple.

6. Implementations